Advanced Data Management: what to state, compute, read and argue

Organised like the paper: A quick checks, B situations, C a real plan, D tickets. green = procedure, red ! = trap, [FS] = on the exam's formula sheet (the last page of the mock). B pages, N rows, r rows per page, M buffer pages (`work_mem`/8 KB), s selectivity, $k = sN$ matching rows, h tree height, W working set, h_{it} hit ratio.

A Quick checks: the fact behind the letter

Who rejects what: grammar: parser ▹ unknown column, type, permission: binder (catalog) ▹ scan or join choice: planner ▹ wrong estimate: statistics (`ANALYZE`) ▹ low hit ratio: buffer pool ▹ deadlock: transaction manager. The planner never runs the query; the executor never chooses.

Pages: record = columns +24 B, rounded up to 8; $r = \lfloor 8168/(\text{record} + 4) \rfloor$; $B = \lceil N/r \rceil$. VACUUM frees space inside pages, the file shrinks only with VACUUM FULL or CLUSTER.

Index facts: lookup = h index pages +1 heap page; h is small (3–4), never $\log_2 N$. An index serves its key expression only (`lower(email)` needs an index on `lower(email)`); a composite (a, b) serves a leading prefix, not b alone; BRIN helps only a physically ordered column; hash helps only =.

Index kinds: **B-tree** (default: =, <, BETWEEN, ORDER BY, LIKE `'abc%'`) ▹ **hash** (= only) ▹ **bitmap index** (one bit vector per value, low cardinality, AND/OR by bit operations; PostgreSQL builds them on the fly from a B-tree: the bitmap scan) ▹ **BRIN** (min/max per block range, tiny, physically ordered columns only) ▹ **GIN/GiST** (jsonb, text, geometry). **Forms**: composite (a, b) leading prefix ▹ expression ▹ partial (`WHERE`) ▹ covering (`INCLUDE`, index-only) ▹ unique = constraint. **Clustered** (heap in key order, one per table; ranges contiguous) vs **secondary** (any number; matches scattered).

Blocking nodes (all input before the first output): Sort, Hash build, HashAggregate, Materialize. **Streaming**: scans, Filter, Limit, Nested Loop, Merge Join, hash probe, GroupAggregate on sorted input. A LIMIT above a breaker saves nothing below it.

Sort: runs vs fan-in $M - 1$ decides the passes [FS]; doubling M removes at most one pass.

Estimate: equality on an MCV value = its frequency; AND multiplies (independence); never add.

Clock-sweep: `usage_count` 0–5, +1 per hit; the hand decrements and evicts at 0; pinned frames are skipped. Pin = in use now; dirty = must be written first.

! Letter without the one-line reason scores one point of three.

B Situations: compute, then decide

Write: **data → formula → numbers with units → the sentence that quotes the number**. Everything marked [FS] is on the formula sheet; below is what is not.

Index or scan, with the correlation

s, k from the predicate [FS] ▹ read `pg_stats.correlation` ▹ heap pages the matches touch:

scattered (corr ≈ 0) : $B\,(1 - (1 - s)^r)$ compute $(1 - s)^r = e^{r \ln(1-s)}$; $sr \geq 3 \Rightarrow$ nearly every page

contiguous (corr $\approx \pm 1$) : $\lceil k/r \rceil \approx sB$ append-only tables, after CLUSTER

▹ index scan cost, assembled from the sheet's parts:

Index Scan $\approx 4h + \lceil k/L \rceil + p_{\text{heap}} \times (4$ scattered $\mid$ 1 contiguous$) + 0.015\,k$ $4h$ descent, leaf pages sequential, p_{heap} from the line above, $0.015k$ = index entry + tuple CPU

Bitmap Scan $\approx 4h + \lceil k/L \rceil + p_{\text{heap}} \times (1 \ldots 4) + 0.015\,k$ same pages, each once in physical order: price 1 when most pages are touched (`Heap Blocks` = p_{heap}); the index scan wins the factor 4 back only when p_{heap} is small

▹ compare with the Seq Scan [FS] ($B + 0.01N + 0.0025N$ per predicate) ▹ between the plain-index threshold and a few percent, a **bitmap scan** reads the same pages once, in order ▹ add the write price: one random page per index per INSERT ▹ decide, quoting both costs. **The one word**: correlation (pages, not rows).

B⁺ tree and latency

reads per lookup = $h + 1$, per range [FS] ▹ time = reads × device (RAM 0.1 μs, SSD 100 μs, HDD 10 ms) ▹ cached levels cost the RAM price ▹ say which target is met and what to buy (SSD, RAM for the index) rather than the pool size.

Sort and "what changes with M"

recompute runs and passes [FS] for each M ▹ one pass needs the input in memory; otherwise at most one pass fewer per doubling ▹ PostgreSQL: `Sort Method` line, `external merge Disk: x` needs `work_mem` $\geq x$, set with `SET LOCAL` in the transaction, never globally.

Join comparison

data B_R, B_S, $|R|$, $|S|$, M, d, indexes, existing order ▹ per algorithm settle the fits case from M first, then its formula [FS] ▹ a sort-merge pays the *full* external sort of each unsorted input ▹ minimum, then the trap answers: rows where pages belong (simple NL), M for $M - 2$ ▹ say when the loser wins: more memory (both read each input once), an index on the inner key with a tiny outer, sorted output wanted ▹ for ORDER BY, aggregate first and sort the groups, not the big input. Each join's disaster: block NL when nothing fits; index NL with a big outer (`loops=` huge); sort-merge with nothing sorted and little memory; hash on <, a build $\gg M$ (`Batches` > 1) or a skewed key.

Buffer pool numbers

EAT = $h_{\text{it}}\,T_{\text{hit}} + (1 - h_{\text{it}})\,T_{\text{miss}}$ then "× RAM" and "× the other scenario"; the miss term dominates

$h_{\text{it}} = \text{hit}/(\text{hit} + \text{read})$ `read` is not disk: the OS cache serves many

ring iff $B >$ shared_buffers/4 frames = `shared_buffers`/8 KB; ring = 32 frames; W survives iff $W \leq$ frames − 32

reload after flooding = $W \times T_{\text{miss}}$ under LRU/clock/FIFO a scan > pool evicts all of W; the scan gains nothing (use-once)

Sizing: `shared_buffers` ≈ 25 % of RAM; size for W, not the database; more starves the OS cache and `work_mem`; measure W (`pg_buffercache`) and h_{it} (`pg_statio_user_tables`) first.

Replacement trace

frames drawn ▹ per request: hit (LRU: to front; FIFO: nothing; clock: set bit) or miss (victim by the rule; dirty victim = an extra write) ▹ count hits, misses, writes ▹ EAT.

! Rows where pages belong. Fan-in M (it is $M - 1$), block NL with M (it is $M - 2$). Reads only (every sort pass reads and writes). Forgetting pass 0. "Index wins because s is small" without pages. "99 % is a good hit ratio" (compute the EAT). Tracing LRU as FIFO. Doubling memory halves the time.

C A real plan: read, locate, fix

Node line: `(cost=start..total rows=est width)` is the planner's belief; `(actual time=first..last rows=act loops=n)` is measured *per loop*: total rows = act × loops, time = last × loops. `start` = work before the first row (a breaker's whole input).

Sub-lines: `Filter` + `Rows Removed by Filter`: tested after reading the row ▹ `Index Cond`: answered inside the index ▹ `Recheck Cond` + `Heap Blocks: exact=`: bitmap scan, pages touched ▹ `Heap Fetches`: index-only, 0 is ideal ▹ `Hash Cond`/`Merge Cond`/`Join Filter` ▹ `Buckets` `Batches` `Memory`: `Batches` > 1 spilled ▹ `Sort Key`, `Sort Method`, `Memory/Disk` ▹ `Group Key` ▹ `Buffers` (per node, children included): `shared hit` = pages found in `shared_buffers`; `read` = pages *not* in the pool, fetched from the OS page cache or the disk (so `read` ≠ disk, and cold/warm runs differ less than expected); `dirtied`/`written` = pages this query modified/flushed; `temp read`/`written` = a sort or hash spilling to temp files.

Tree: indentation = children; first child = outer or probe, second = inner or build (under `Hash`); execution starts at the leftmost leaf of the deepest breaker. **Reconstruct the query**: Scan on `t a` = FROM; `Hash`/`Merge Cond` = ON; `Filter`/`Index Cond` = WHERE; `Group Key` = GROUP BY; `Sort Key` = ORDER BY; `Limit` = LIMIT.

Five moves

1 leaves: scan type, filter, est vs act ▹ 2 up: join algorithm, order, aggregate, sort ▹ 3 the *lowest* node with a large q-error is the cause; everything above inherits it ▹ 4 resources: `Buffers`, `Sort Method`, `Batches`, `loops` of the inner side, `Heap Blocks` ▹ 5 decide: wrong plan from a wrong estimate (statistics fix) / right plan but slow (memory, index, cold cache, CPU-bound) / nothing to fix; name the command.

Where an estimate comes from

scan `rows=` = MCV frequency (or $1/d$, histogram) $\times N$ ▹ a `Filter` multiplies by its own selectivity (independence) ▹ join `rows=` [FS] ▹ q-error = max(est/act, act/est); ≥ 10 is the cause; an *under*-estimate buys per-row plans (nested loop, index lookups) for a big input ▹ name the failure and the command:

```
stale: n_live_tup <> reltuples, old
  last_analyze -> ANALYZE t;
correlated columns (a determines b;
  ANALYZE does not help) -> CREATE
  STATISTICS s (dependencies) ON a, b
  FROM t; ANALYZE t;
skew (rare value estimated at 1/d) ->
  ALTER TABLE t ALTER COLUMN c SET
  STATISTICS 1000; ANALYZE t;
```

Where the time goes

`Heap Blocks` = the scattered-pages formula on B ▹ inner side: pages per loop $(h + 1) \times$ `loops` = `hit` + `read` ▹ seconds ≈ `read` × the random price (50–100 μs) ▹ compare the node's `actual time` spans to see which node owns the seconds ▹ the overall hit ratio hides a hot node: read `Buffers` per node.

What each fix changes

extended statistics: estimate right → the join and its order change (hash join, build on the small filtered side) ▹ **ANALYZE**: only stale counts ▹ **composite index**: `Filter` moves into `Index Cond`; `Heap Blocks`, estimate and time unchanged if the filter removed nothing ▹ **CLUSTER**: correlation → 1, `Heap Blocks` → $\lceil k/r \rceil$, scan time collapses, the join cost stays; one-off rewrite under lock, decays with new rows ▹ **covering index** (INCLUDE): `Heap Fetches: 0`, no heap pages ▹ **partial index**: tiny, only when the query repeats the predicate ▹ **work_mem**: `Disk`/`Batches` disappear ▹ **hash join alternative**: build on the smaller *filtered* input (fits in `work_mem`: `Batches: 1`), probe with a sequential scan: sequential pages instead of `loops` random ones.

! Fixing the scan when the join owns the time. "`read` means disk". Inner `rows` read as a total. Blaming the top node. Adding an index for a 10× estimate error. Calling a bitmap heap scan blocking (it streams) or a bitmap index scan streaming (it does not).

D Tickets: claim, mechanism, number

Half a page: **claim** (right, wrong, or both)

▹ **mechanism** (which subsystem, which rule) ▹ **one number** with units ▹ **consequence** and what to do instead ▹ the tempting wrong answer. The arguments the course expects:

"Column store, analytics are slow": a scan of c of k columns reads c/k of the bytes, compressed further; an insert writes k places instead of 1, a point read reassembles from k. Both workloads exist: keep the row store for the app, give analytics a column copy (COPY to Parquet + DuckDB) or, inside PostgreSQL, BRIN on the time column and a covering index.

"LRU is optimal, replace clock-sweep": OPT (evict the furthest next use) is optimal and needs the future; LRU approximates it with the past; exact LRU costs a list move under a lock on *every* hit, which at 99 % hits is the bottleneck; clock sets a bit, clock-sweep adds frequency via `usage_count`; all of them fail on a scan larger than the pool, which PostgreSQL fixes by intent (rings), not by policy.

"Freeze the join order" (`join_collapse_limit = 1`): n tables have $n!$ left-deep orders ($9! = 362{,}880$) but System-R DP costs $2^n - 1$ subsets (511) plus interesting orders; the written order ignores selection pushdown (the most filtered input should come first) and goes stale with the data; fix the one query with `SET LOCAL`, a prepared statement or a materialised intermediate, never server-wide.

"Doubling work_mem halves the sort": passes are a ceiling of a logarithm; a doubling removes at most one merge pass; only $M \geq B$ makes it one pass; find the `Disk` figure and give exactly that.

"Hash joins are buggy, switched off": `enable_*` is a 10^{10} cost penalty applied to every query; with memory both algorithms read each input once and tie; a correctness bug is reproduced and filed, not planned around.

"An index on a small filter never helps / always helps": pages, not rows: scattered matches touch $B(1-(1-s)^r)$ pages at the random price, contiguous ones sB; read the correlation; a bitmap scan sits between; every index costs a random write per insert; drop `idx_scan = 0`.

"More shared_buffers": a use-once scan has hit ratio 0 at any size and is ringed anyway; the pool should hold W, which is measured, not guessed; more pool starves the OS cache (which serves `reads`) and makes checkpoints bursty.

"The slow morning / the nightly scan": flooding under LRU reloads W at T_{miss} each; with rings W survives and the cause is elsewhere (OS cache, a COPY ring, a checkpoint): measure with `pg_buffercache` after the job.

"Statistics are fresh, so the estimate is right": fresh per-column frequencies still multiply under independence; correlated columns need `CREATE STATISTICS`.

"The second run is fast, so the query is fine": the cache, not the plan; compare `Buffers` hit and read; all hits and still slow = CPU-bound, needs a better plan, not memory.

"Commit wrote the data" (steal, force, WAL): **steal** = a dirty page of an *uncommitted* transaction may be written to disk (the pool needs the frame) ⇒ needs **undo**: the log keeps the before-image. **Force** = commit must write all its dirty pages (slow, random); **no-force** = it need not ⇒ needs **redo**: the log keeps the after-image. Real systems: steal + no-force + **WAL**: the log record is written before its data page and the log is forced at commit (one sequential append); recovery redoes committed changes, undoes uncommitted ones. Committed data is safe in the log before it is in the data file.

! A ticket answered without a number. Agreeing or refusing without the mechanism. Not saying what to do instead.

Commands to name

```
EXPLAIN (ANALYZE, BUFFERS) q; -- DML:
  BEGIN; ... ROLLBACK;
SELECT attname, n_distinct,
  most_common_vals, most_common_freqs,
  histogram_bounds, correlation FROM
  pg_stats WHERE tablename = 't';
SELECT reltuples, relpages FROM pg_class
  WHERE relname = 't'; SELECT
  pg_relation_size('t') / 8192;
ANALYZE t; CREATE STATISTICS s
  (dependencies) ON a, b FROM t; ANALYZE
  t; ALTER TABLE t ALTER COLUMN c SET
  STATISTICS 1000;
BEGIN; SET LOCAL work_mem = '256MB';
  ...; COMMIT; SET enable_hashjoin =
  off; RESET ALL;
CREATE INDEX ON t (a, b); ON t
  (lower(email)); ON t (a) WHERE status
  = 'open'; ON t (a) INCLUDE (b); ON t
  USING brin (ts); CLUSTER t USING idx;
  DROP INDEX i;
SELECT indexrelname, idx_scan FROM
  pg_stat_user_indexes WHERE relname =
  't'; SELECT heap_blks_hit,
  heap_blks_read FROM
  pg_statio_user_tables WHERE relname =
  't';
CREATE EXTENSION pg_buffercache; SELECT
  c.relname, count(*) FROM
  pg_buffercache b JOIN pg_class c ON
  b.relfilenode =
  pg_relation_filenode(c.oid) GROUP BY 1
  ORDER BY 2 DESC;
```

Test data (one `random()` draw per row in the subquery, then several cuts; `ANALYZE` after):

```
INSERT INTO e (uid, ts, kind, amt)
  SELECT 1 + (random() * 999999)::int,
  now() - random() * interval '365
  days', CASE WHEN r < 0.9 THEN 'view'
  WHEN r < 0.99 THEN 'cart' ELSE 'buy'
  END, round((5 + random() *
  100)::numeric, 2) FROM (SELECT
  random() AS r FROM generate_series(1,
  1000000)) g; ANALYZE e;
generate_series(1, n) AS i; i % 10
  (cycling); (random() * n)::int
  (uniform); floor(-ln(random()) * m)
  (long tail); 'x' || i, md5(i::text)
  (strings); now() - (i || '
  s')::interval (ordered ts, correlation
  1)
```

! `random()` inside each WHEN is a new draw per branch: wrong proportions. Forgetting `ANALYZE` after the load: the planner still believes the table is empty.

From the query-cost sheet, NOT on the exam's fo

Costs are page I/Os; writing the final output is never counted. **Any operator**: write its steps, count pages read and written and rows handled, multiply by the prices, add up.

col = $v \notin$ MCV : $s = \dfrac{1 - \text{null_frac} - \sum \text{MCV freqs}}{n_{\text{distinct}} - \#\text{MCV}}$

no MCV list: $s = (1 - \text{null_frac})/n_{\text{distinct}}$

col $< c$: $s = \sum_{\text{MCV } v<c} \text{freq}(v) + \text{frac}(c)\,(1 - \text{null_frac} - \sum \text{MCV freqs})$

frac(c) = (whole buckets below c + linear share of c's bucket)/#buckets BETWEEN a, b: frac(b)−frac(a)

$s_{\neg A} = 1 - s_A$; NULLs satisfy no comparison

foreign-key join: $|R \bowtie S| =$ $|R|$ (the referencing side); $m = |S|/d_S$ $m =$ matches per outer row

#leaves = $\lceil N/L \rceil$, $h =$ $1 + \lceil \log_F \#\text{leaves} \rceil$, check $F^{h-1}L \geq N$

heap(m) = m unclustered, $\lceil m/r \rceil$ clustered, 0 index-only

sort: $B \leq M \Rightarrow$ cost B; $\lceil B/M \rceil \leq M - 1 \Rightarrow$ 2 passes, $4B$ $M \gtrsim \sqrt{B}$

simple NL $B_R + |R|\,B_S$; page NL $B_R + B_R B_S$

block NL, outer fits ($B_R \leq M - 2$) : $B_R + B_S$

sort-merge, merge in the last sort pass: $3(B_R + B_S)$ Grace needs $M \gtrsim \sqrt{B_R}$

Where each comes from: block NL: $M - 2$ buffers hold a chunk of R, one holds the S page, one the output; one scan of S per chunk ▹ index NL: read R once; per row descend the index (h) and fetch its m matches ▹ sort-merge: sort both, then one merge pass reading each once ▹ hash, fits: build on R in memory, stream S, nothing written ▹ Grace: partition both with h_1 into $M - 1$ buckets (read + write both), then per bucket pair build and probe (read both): $3(B_R + B_S)$. **Choosing**: smaller table as outer or build ▹ one side fits ⇒ hash or block NL, $B_R + B_S$ ▹ few outer rows and a big indexed inner ⇒ index NL ▹ equality only ⇒ hash ▹ ranges or sorted output needed ⇒ sort-merge.

filter, projection, hash aggregate (groups fit): 0 beyond reading the input

sort aggregate, DISTINCT, ORDER BY: sort(R) or 0 if the input is already sorted

Seq Scan = $B + 0.01\,|R| + 0.0025\,q\,|R|$ q conditions

hash join CPU = $0.0025\,(|R| + |S|) + 0.01\,|\text{out}|$

index NL, per outer row ≈ $4h + 4m + 0.015\,m$ (unclustered)

Pricing a plan: statistics → selectivity → rows → pages per node, bottom-up ▹ per node its formula with the fits case set by M ▹ CPU terms if asked ▹ sum; keep the cheapest ▹ bad estimates propagate up and multiply: check with EXPLAIN ANALYZE.

What the names mean

EXPLAIN: `ANALYZE` runs it (actual rows, times, loops); `BUFFERS` adds hit/read per node. `pg_stats` (one row per column): `attname` column name ▹ `null_frac` fraction of NULLs ▹ `n_distinct` distinct values (negative = fraction of the rows, e.g. −1 a key) ▹ `most_common_vals` / `most_common_freqs` the MCV list and each value's fraction of rows ▹ `histogram_bounds` boundaries of equal-depth buckets of the non-MCV values ▹ `correlation` −1..1, physical order versus value order. `pg_class`: `reltuples`, `relpages` the row and page counts the planner believes (as of the last ANALYZE/VACUUM). `pg_relation_size('t')` real bytes of the heap file (/8192 = pages); `pg_total_relation_size` adds indexes and TOAST. **Statistics commands**: `ANALYZE t` resamples; `CREATE STATISTICS .. (dependencies) ON a, b` measures how much `a` determines `b`; `SET STATISTICS 1000` lengthens the MCV list and histogram for that column (default 100). **Settings**: `work_mem` memory per sort or hash node per connection (`SET LOCAL`: this transaction only); `shared_buffers` the pool size; `enable_hashjoin = off` a cost penalty, not a ban; `RESET ALL` back to defaults. **Index forms**: `(a, b)` composite; `(lower(email))` expression; `WHERE ..` partial; `INCLUDE (b)` covering; `USING brin` block-range min/max; `CLUSTER t USING idx` rewrites the table in that index's order. `pg_stat_user_indexes`: `indexrelname` index name, `idx_scan` times the index was used since the stats reset. `pg_statio_user_tables`: `heap_blks_hit` pages found in `shared_buffers`, `heap_blks_read` pages requested from the OS or disk; `n_live_tup`, `n_dead_tup`, `last_analyze` in `pg_stat_user_tables`. `pg_buffercache`: one row per frame of the pool; `relfilenode` the file id of the relation held in the frame, joined to `pg_relation_filenode(c.oid)` to get its name; `isdirty`, `usagecount` the frame's dirty bit and clock-sweep count.